

Introduction to Deep Learning

Unit 1: 01AML307 Deep Learning

Mr. Deepak Dilip Mahajan,
Asst.Professor, Department of CSE(AIML),
DKTE Society's Textile & Engineering Institute, Ichalkaranji

Lecture 01: Introduction to Deep Learning

1. Introduction to Deep Learning
2. A Brief History
3. Need of Deep Learning
4. Applications of Deep Learning
5. Neural Networks and their Architectures
6. Activation Functions and their Properties
7. Practice exercises and discussions: Building ANN for Business Problem

1. Introduction to Deep Learning

Deep Learning (DL) is a specialized and advanced subfield of Machine Learning (ML), which itself is a branch of Artificial Intelligence (AI).

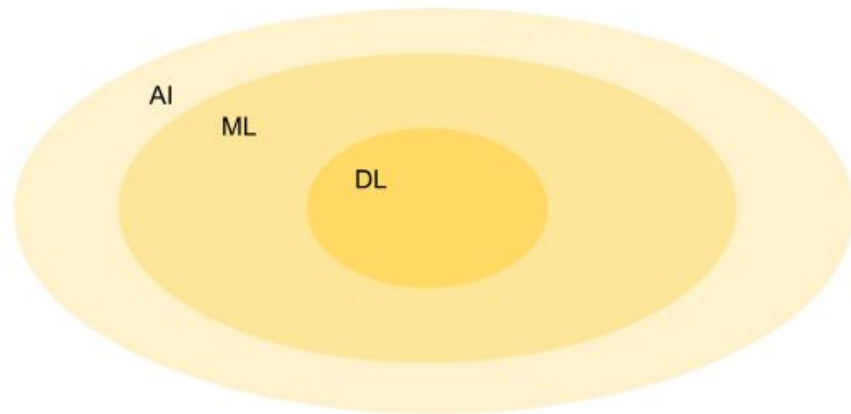
At its core, Deep Learning is about enabling computers to learn and make decisions by analyzing vast amounts of data, much like the human brain learns from experience.

Deep $\Rightarrow$ Depth of Neural Network

Shallow Networks: Have only one or two hidden layers

Deep Networks: Have dozens or even hundreds of layers

Image : Pixel $\rightarrow$ Pattern $\rightarrow$ Edges $\rightarrow$ Object



2. A Brief History of Deep Learning

1943: Computational Model

1958: Perceptron

1986: Backpropagation Algorithm : [Backpropagation Paper](#)

1989: LeNet-5 Paper : [LeNet-5 Paper](#)

2006: Greedy Layer-wise Training, Rise of GPU, More Data : [Greedy Layer-wise Training Paper](#)

2012: AlexNet - ImageNet : [ImageNet Paper](#)

.....: Open Source Frameworks - Tensorflow, PyTorch, Keras

.....: AI Democratization

3. Need for Deep Learning

- Unstructured Data
- Scalability
- Complex Patterns
- Utilize Computational Power
- Overcoming Limitations of Shallow Networks

3. Need: How it is differ from ML?

- Relationship
- Core Concept
- Feature Engineering
- Data Requirements
- Model Complexity
- Computational Power
- Training Time
- Interpretability
- Problem Solving
- Use Cases
- Accuracy
- Human Intervention

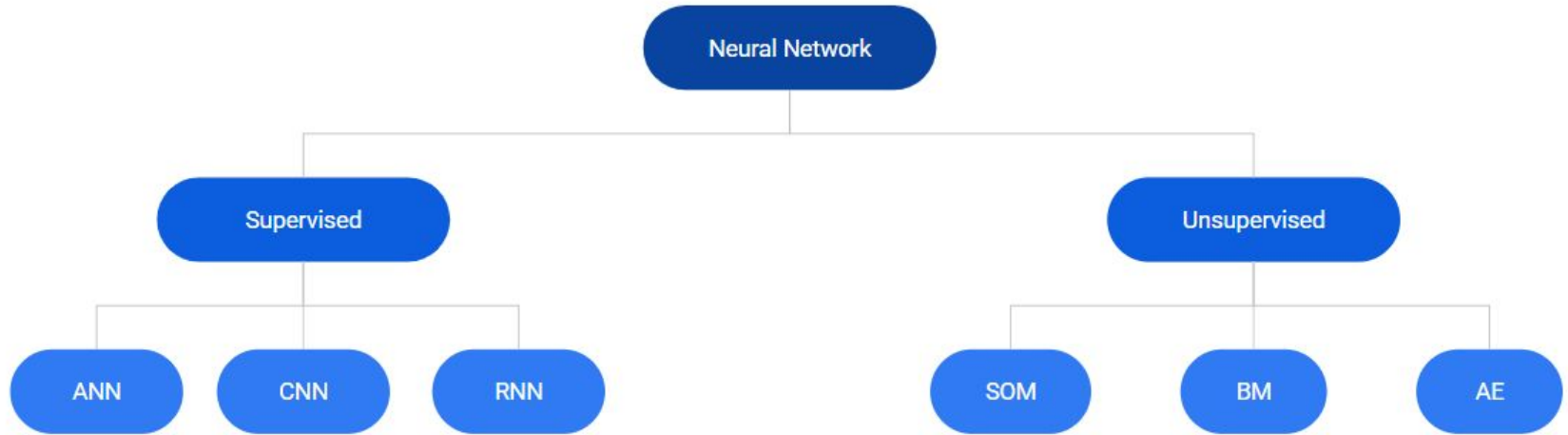
4. Applications of Deep Learning

- **Image Recognition & Computer Vision:**
 - **Facial Recognition:** Unlocking your phone, security cameras.
 - **Object Detection:** Self-driving cars identifying pedestrians, traffic signs, other vehicles.
 - **Medical Imaging Analysis:** Detecting diseases (e.g., cancer, tumors) from X-rays, MRIs, CT scans with high accuracy.
 - **Image Classification:** Categorizing images (e.g., tagging photos on social media).
- **Natural Language Processing (NLP):**
 - **Machine Translation:** Google Translate, automatically translating text between languages.
 - **Sentiment Analysis:** Determining the emotional tone of text (e.g., customer reviews).
 - **Chatbots & Virtual Assistants:** Siri, Alexa, Google Assistant, customer service bots.
 - **Text Summarization:** Automatically generating summaries of long documents.
- **Speech Recognition:**
 - **Voice Commands:** Interacting with smart devices, dictating messages.
 - **Speech-to-Text:** Transcribing spoken words into written text.
 - **Speaker Identification:** Recognizing who is speaking.

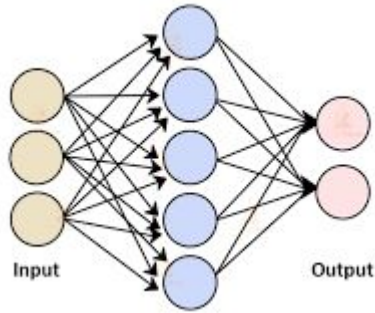
4. Applications of Deep Learning

- **Generative AI:**
 - **Image Generation:** Creating realistic images from text descriptions (e.g., Mid-journey, DALL-E).
 - **Text Generation:** Writing articles, stories, code, or even poems (e.g., ChatGPT, Bard).
 - **Music Composition:** Generating new musical pieces.
 - **Video Generation:** Creating short video clips from text or images.
- **Recommendation Systems:**
 - **E-commerce:** Recommending products on Amazon, Flipkart.
 - **Streaming Services:** Suggesting movies on Netflix, songs on Spotify.
- **Autonomous Vehicles:**
 - Perception (understanding the environment), planning, and control in self-driving cars.
- **Drug Discovery & Healthcare:**
 - Predicting molecular properties, accelerating drug discovery, designing new proteins.
- **Fraud Detection:**
 - Identifying anomalous transactions in banking and finance.
- **Gaming:**
 - Creating more intelligent NPCs (Non-Player Characters) and improving game realism.

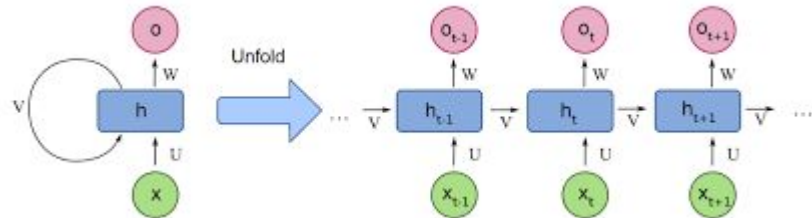
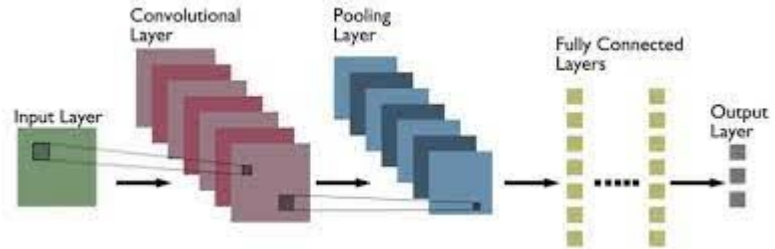
5. Neural Network: Architectures



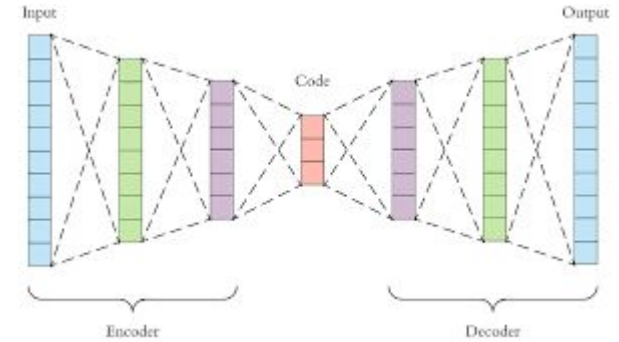
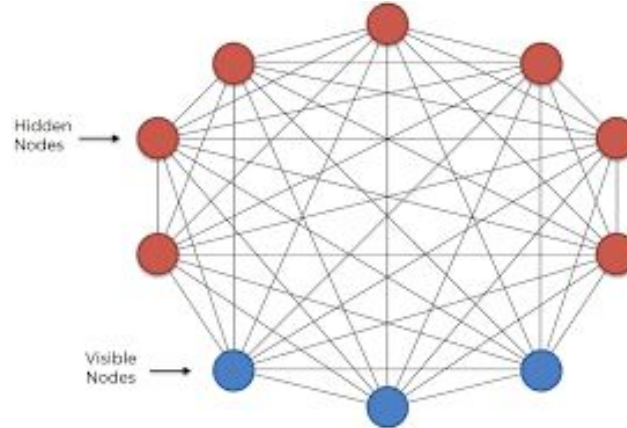
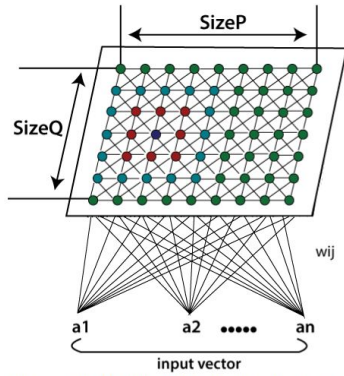
Supervised NN



- Description
- Architecture
- Use Cases



Unsupervised NN



- Description
- Architecture
- Use Cases

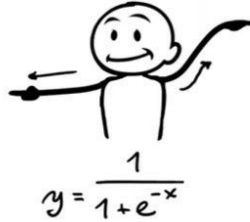
Performance of Neural Network

1. Activation Function
2. Weight Initialization
3. Batch Normalization
4. Optimizer

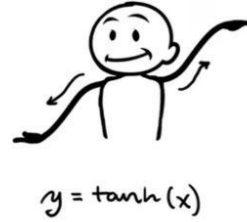
Activation Function

- Providing Non-linearity
- Restricting Output Values
- Choices:
 - Sigmoid
 - Tan-h
 - ReLU
 - Leaky ReLU
 - Softmax

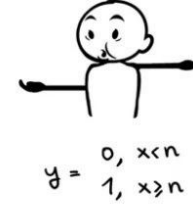
Sigmoid



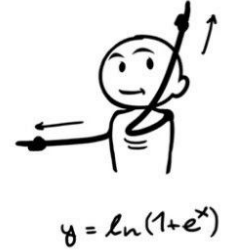
Tanh



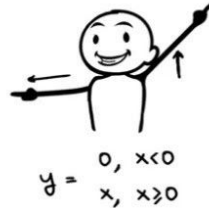
Step Function



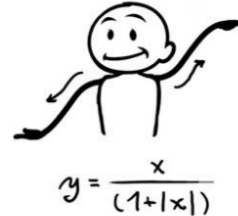
Softplus



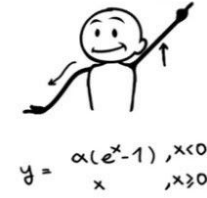
ReLU



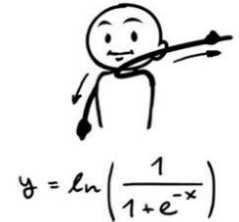
Softsign



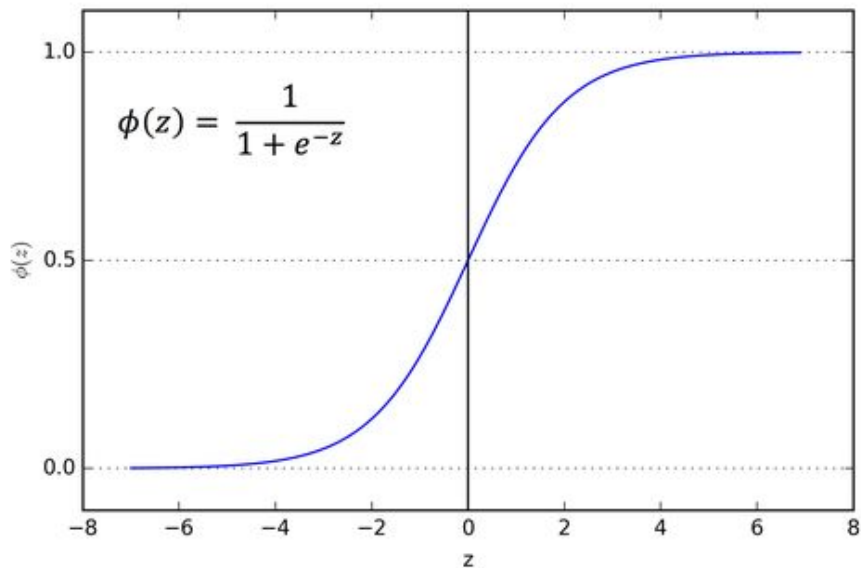
ELU



Log of Sigmoid



Sigmoid



Equation:

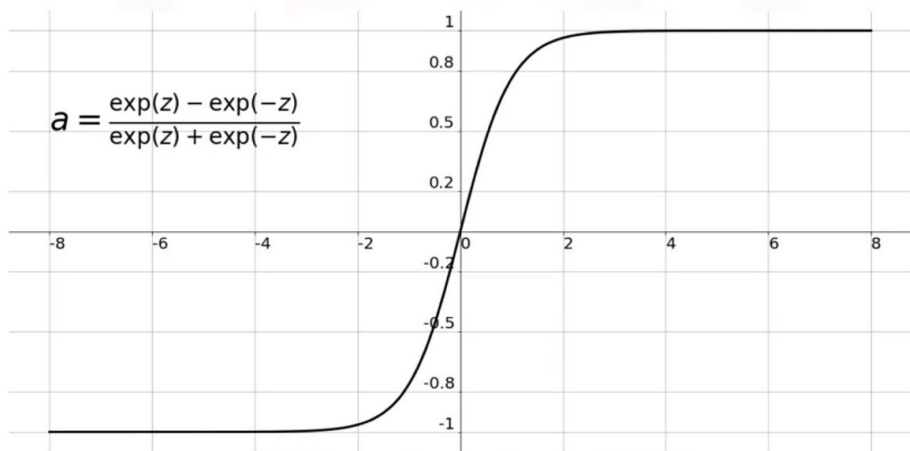
Graph: An 'S'-shaped curve.

Output Range: (0,1)

Key Properties & Characteristics:

- Squashes Output: YES
- Non-Linear: YES
- Differentiable: YES
- Vanishing Gradient Problem: YES
- Not Zero-Centered
- When to Use:

Tan-h



Equation:

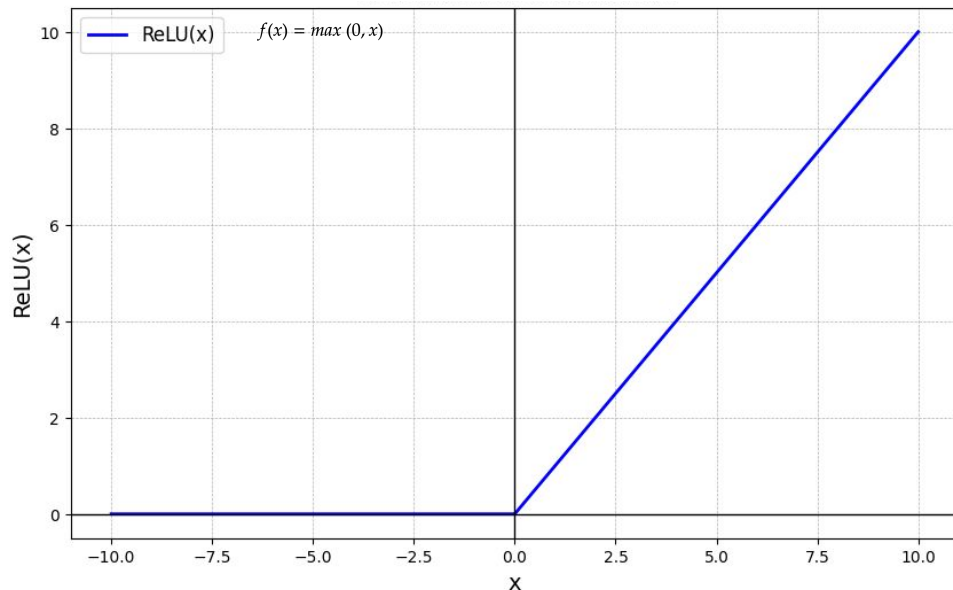
Graph: An 'S'-shaped curve.

Output Range: (0,1)

Key Properties & Characteristics:

- Squashes Output: YES
- Non-Linear: YES
- Differentiable: YES
- Vanishing Gradient Problem: YES
- Zero-Centered
- When to Use: in hidden layers, and often performs better than sigmoid due to its zero-centered output

ReLU



Equation:

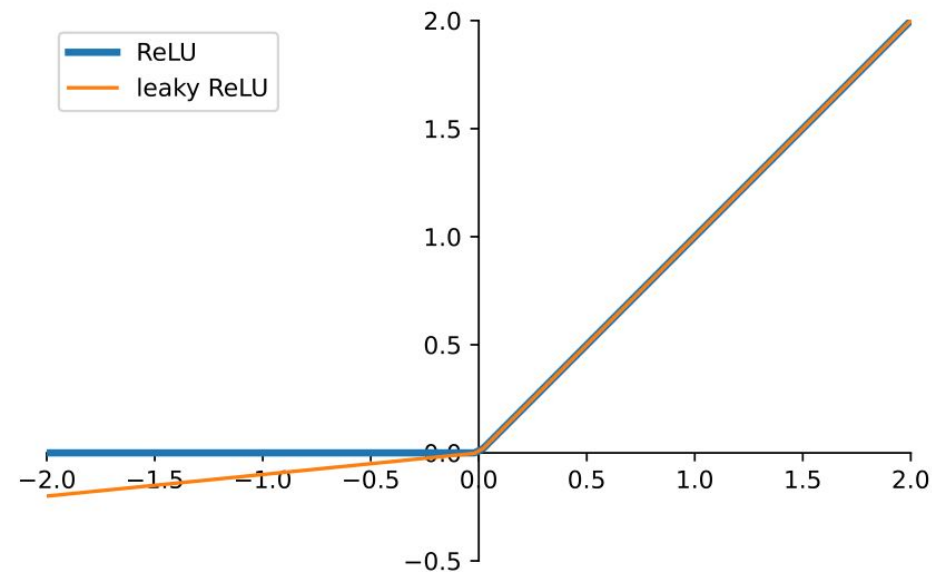
Graph:

Output Range: $(0, \infty)$

Key Properties & Characteristics:

- Non-Linear: non-linear due to the "bend" at zero.
- Computational Efficiency
- Solves Vanishing Gradient Problem: (for +ve)
- Sparsity
- Dying ReLU Problem
- When to Use: Hidden layers of modern DL Models (in most of the CNN)

Leaky ReLU



Equation:
$$f(x) = \begin{cases} x & \text{if } x > 0 \\ \alpha x & \text{if } x \leq 0 \end{cases}$$

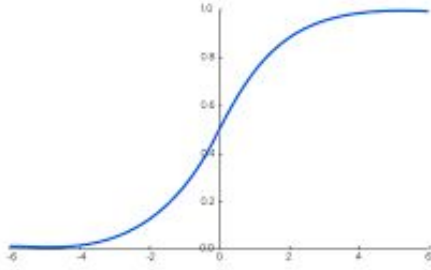
Graph:

Output Range: $(-\infty, \infty)$

Key Properties & Characteristics:

- Nonlinear and Differentiable
- Maintain advantages of ReLU
- Address Dying ReLU Problem
- Performance Improvement
- When to Use: A good alternative to ReLU for hidden layers, to overcome Dying ReLU problem

Softmax



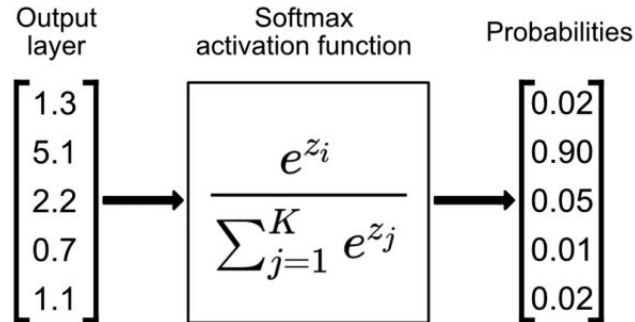
Equation:

Graph:

Output Range: (0,1) for each individual output, and the sum of all outputs equals 1.

Key Properties & Characteristics:

- Transforms to Probability Distributions
- Essential for tasks where an input belongs to one and only one of several possible classes.
- Sensitive to largest input
- When to Use: output layer for multi-class classification problems.



Activation Functions Summary

Activation Function	Output Range	Zero-Centered?	Vanishing Gradient	Dying ReLU	Computation	Common Use
Sigmoid	$(0, 1)$	No	Yes	No	Medium	Binary Class. Output Layer
Tanh	$(-1, 1)$	Yes	Yes	No	Medium	Hidden Layers (less common now)
ReLU	$[0, \infty)$	No	No (for positive)	Yes	Fast	Hidden Layers (most common)
Leaky ReLU	$(-\infty, \infty)$	No	No (for positive)	No	Fast	Hidden Layers (alternative to ReLU)
Softmax	$(0, 1)$ per output, Sum=1	N/A (across vector)	Yes (can be an issue for very large inputs)	N/A	Medium	Multi-Class. Output Layer

Activation Function

Roll in ML :

- Linear Regression: ?
- Logistic Regression: ?
- Tree Based Models: ?
- SVM: ?
- KNN: ?
- NOT Universally Applied

Roll in DL :

- Here Activation Functions are indispensable due to deep network.
- Enable Learning of Complex Patterns
- Facilitating Gradient Flow
- Controlling Output Range and Sparsity
- Universally Applied

Lecture 02: Weight Initialization & Basic Optimizers

1. Vanishing / Exploding Gradients
2. Weight Initialization Strategies:
 - a. Xavier
 - b. He Init
3. Basic Optimization techniques:
 - a. Gradient Descent
 - b. SGD
 - c. Mini-batch

Vanishing / Exploding Gradients

A. Vanishing Gradients

This occurs when the gradients become insignificantly small as they travel backward from the output layer to the input layer.

The Cause: ?

[Result](#)

B. Exploding Gradients

This is the opposite problem, where gradients accumulate and become massive.

The Cause: ?

[Result](#)

Weight Initialization Strategies

Why Not Zero ? → Symmetry Problem , to break this symmetry -

Strategy 1: Xavier (Glorot) Initialization

- Designed specifically for Sigmoid or Tanh activation functions
- It ensures that the variance of the activations stays the same across every layer
- It prevents the signal from dying out (vanishing) or blowing up (exploding)
- The Formula: Weights are drawn from a distribution with variance (code)

$$\text{Var}(W) = \frac{2}{n_{in} + n_{out}}$$

(Where n_{in} is the number of input neurons and n_{out} is the number of output neurons).

Weight Initialization Strategies

Strategy 2: He Initialization (Kaiming Init)

- Designed specifically for ReLU activation functions (*Why not Xavier for ReLU?*)
- ReLU kills half of the variance
- He initialization compensates for this by doubling the variance compared to a standard initialization: ([code](#))

$$\text{Var}(W) = \frac{2}{n_{in}}$$

(Notice we only divide by inputs n_{in} , making the weights slightly larger to maintain signal strength through ReLU layers).

Feature	Xavier (Glorot) Init	He (Kaiming) Init
Best Used With	Sigmoid, Tanh	ReLU, Leaky ReLU
Problem Solved	Keeps variance constant for S-shaped functions	Compensates for ReLU "zeroing out" negative values
Key Math	$Var(W) = \frac{2}{n_{in} + n_{out}}$	$Var(W) = \frac{2}{n_{in}}$

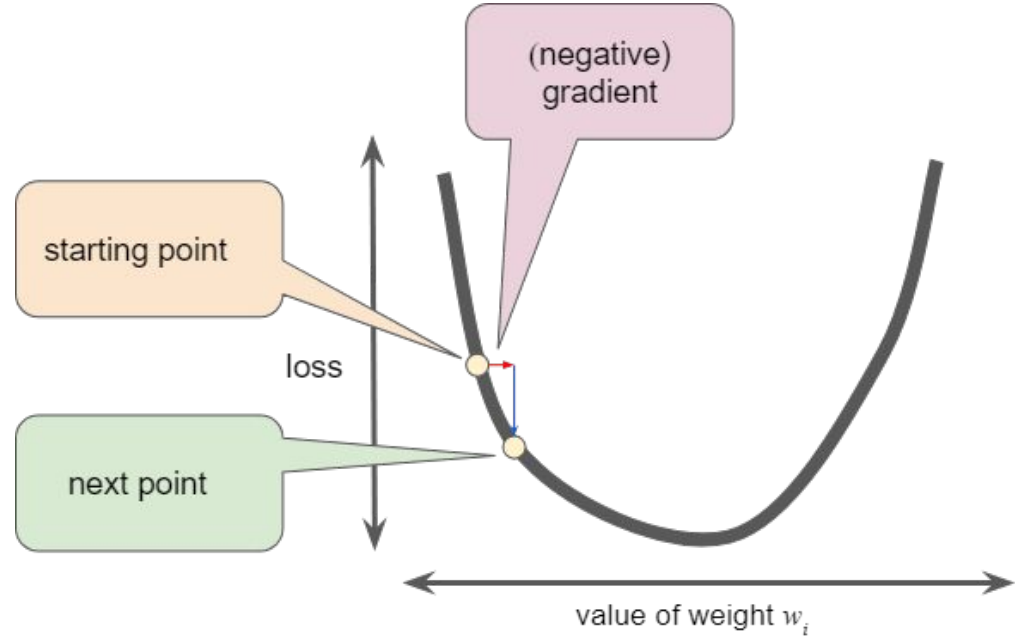
Optimization Algorithms: “Engine” of Deep Learning

- Batch Gradient Descent
- Stochastic Gradient Descent
- Mini-Batch Gradient Descent

Gradient Descent

Calculating the loss function for every conceivable value of

Over the entire data set would be an inefficient way of finding the convergence point. Let's examine a better mechanism—very popular in machine learning—called gradient descent.



Gradient Descent (Batch Gradient Descent)

How it works:

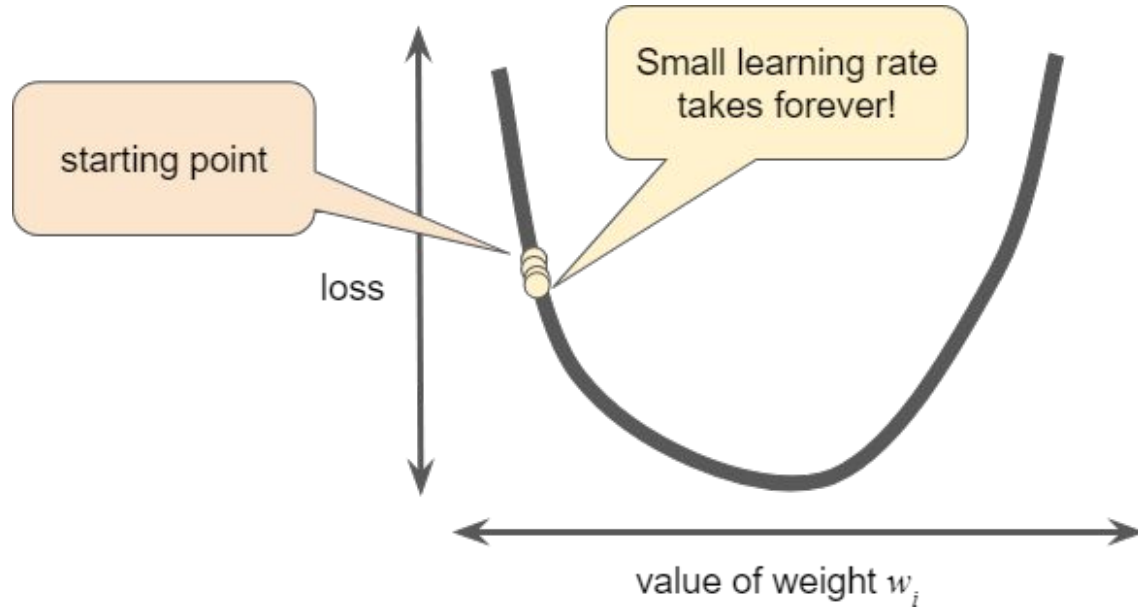
- Passes the **entire dataset** (all training examples) through the network.
- Calculates the average error (loss) for the whole dataset.
- Calculates the gradient (slope) of that error.
- Updates the weights **once**.
- **The Analogy:** Imagine you are on a mountain and want to get to the bottom. In Batch Gradient Descent, you stop, look at every single blade of grass and rock on the entire mountain to map the perfect path, take **one precise step**, and then repeat the process.
- **Pros:**
 - **Stable convergence:** The path to the minimum is smooth and direct because the error calculation is exact (based on all data).
- **Cons:**
 - **Extremely Slow:** If you have 1,00,000 images, you have to process all 1,00,000 just to make a *single* update to the weights.
 - **Memory Heavy:** Storing gradients for the whole dataset can exceed RAM.

Learning Rate

The gradient vector has both a direction and a magnitude. Gradient descent algorithms multiply the gradient by a scalar known as the **learning rate** (also sometimes called step size) to determine the next point.

Hyperparameters are the knobs that programmers tweak in machine learning algorithms. Most machine learning programmers spend a fair amount of time tuning the learning rate. If you pick a learning rate that is too small, learning will take too long

[Try This](#)



Single Step in Gradient Descent

$$\underbrace{\Delta_w(t)}_{\substack{\text{Gradient} \\ \text{Current Iteration}}} = \underbrace{-\varepsilon}_{\text{Bias}} \underbrace{\frac{dE}{dw(t)}}_{\substack{\text{Error} \\ \text{Weight vector}}} + \underbrace{\alpha \Delta_w(t-1)}_{\substack{\text{Learning Rate} \\ \text{Gradient} \\ \text{Previous Iteration}}}$$

Batch Gradient Descent (BGD)

This method considers the entire dataset (all data points) to calculate the gradient and update the model's parameters (weights and biases) in each iteration. While it converges to a minimum, it can be slow for very large datasets because of the massive computations involved.

$$\theta = \theta - \eta * \nabla_{\theta} J(\theta)$$

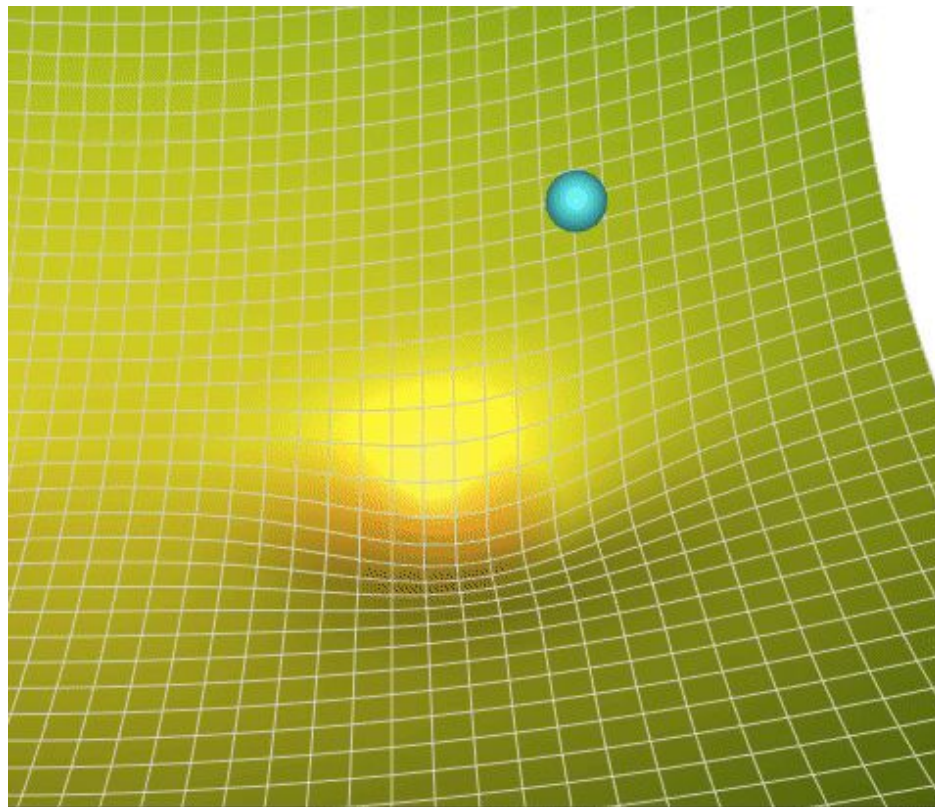
where:

θ (theta) represents the vector of model parameters (weights and biases) you want to update.

η (eta) is the learning rate, a hyperparameter that controls the step size of the update.

$\nabla_{\theta} J(\theta)$ (nabla theta J(theta)) is the gradient of the cost function $J(\theta)$ with respect to the parameter vector θ . This is a vector containing the partial derivative of the cost function for each parameter.

Working of Gradient Descent



Issues with Gradient Descent

1. Fixed Learning Rate

Solution → Learning Rate Scheduler, but need to be predefined ...

2. Same Learning Rate for Multiple Weights (all directions)

Convex vs Non-Convex → Contour Graph will help you to understand this...

3. Local Minima (Noisy Gradient)

4. Saddle Point (Consistent Gradient)

5. High Curvature

So we need new optimizers...

Stochastic Gradient Descent (SGD)

"Stochastic" means random. This approach goes to the extreme opposite of Batch Gradient Descent.

- **How it works:**
 - Pick a **single random training example**.
 - Calculate the error for that *one* example.
 - Update the weights immediately.
 - Repeat for the next random example.
- **The Analogy:** You are on the same mountain. Instead of mapping the whole terrain, you look at the ground right under your feet, feel the slope, and take a quick step. You might stumble or go slightly uphill occasionally, but you move very fast.
- **Pros:**
 - **Fast Learning:** Weights are updated millions of times per epoch (pass through the dataset) rather than just once.
 - **Escaping Local Minima:** Because the path is "noisy" and erratic, it can jump out of shallow holes (local minima) that might trap Batch Gradient Descent.
- **Cons:**
 - **Noisy convergence:** The error curve will zigzag wildly. It might never settle on the exact minimum but will hover around it.
 - **Inefficient:** It doesn't utilize vectorization (doing math on large matrices at once) effectively because it processes items one by one.

Stochastic Gradient Descent

In gradient descent, a **batch** is the set of examples you use to calculate the gradient in a single training iteration.

A very large batch may cause even a single iteration to take a very long time to compute.

Large Dataset $\propto$ Redundant Data , and some redundancy is good also, but enormous batches tend **NOT** to carry much more predictive value

This approach **works on a single data point** at a time, making it faster for large datasets. However, the updates can be noisy due to the limited information used, **potentially leading to zig-zagging behavior** during convergence.

Mini-Batch Gradient Descent

This is the **Goldilocks solution**—the standard industry practice used in almost all Deep Learning today. It combines the best of both worlds.

- **How it works:**
 - Split the data into small batches (commonly 32, 64, or 128 samples).
 - Pass **one batch** through the network.
 - Calculate the average error for that batch.
 - Update the weights.
- **The Analogy:** You take a small group of friends with you down the mountain. You ask them "Which way is down?" and take the average of their answers. It's faster than asking everyone on earth (Batch), but more accurate than just trusting your own feet (SGD).
- **Pros:**
 - **Computational Efficiency:** It maximizes GPU usage by using vectorization (processing 64 images at once is much faster for a GPU than processing 1 image 64 times).
 - **Stability vs. Speed:** It provides a smoother convergence than SGD but is much faster than Batch.

Mini-Batch Stochastic Gradient Descent (MB — SGD)

This strikes a balance between BGD and SGD.

It processes the data in small chunks called mini-batches (groups of data points).

The gradient is calculated for each mini-batch, and the model parameters are updated accordingly.

Feature	Batch Gradient Descent	Stochastic Gradient Descent (SGD)	Mini-Batch Gradient Descent
Data used per step	Entire Dataset	1 Example	Batch Size (e.g., 32, 64)
Update Frequency	Once per epoch	N times per epoch	N / BatchSize times per epoch
Speed per step	Very Slow	Very Fast	Fast
Convergence Path	Smooth, straight line	Noisy, Zig-zag	Slightly wobbly but direct
Common Usage	Theoretical / Small data	Online learning	Standard for Deep Learning

$$\theta_{t+1} = \theta_t + \eta \cdot \nabla \theta_t$$

[code](#)

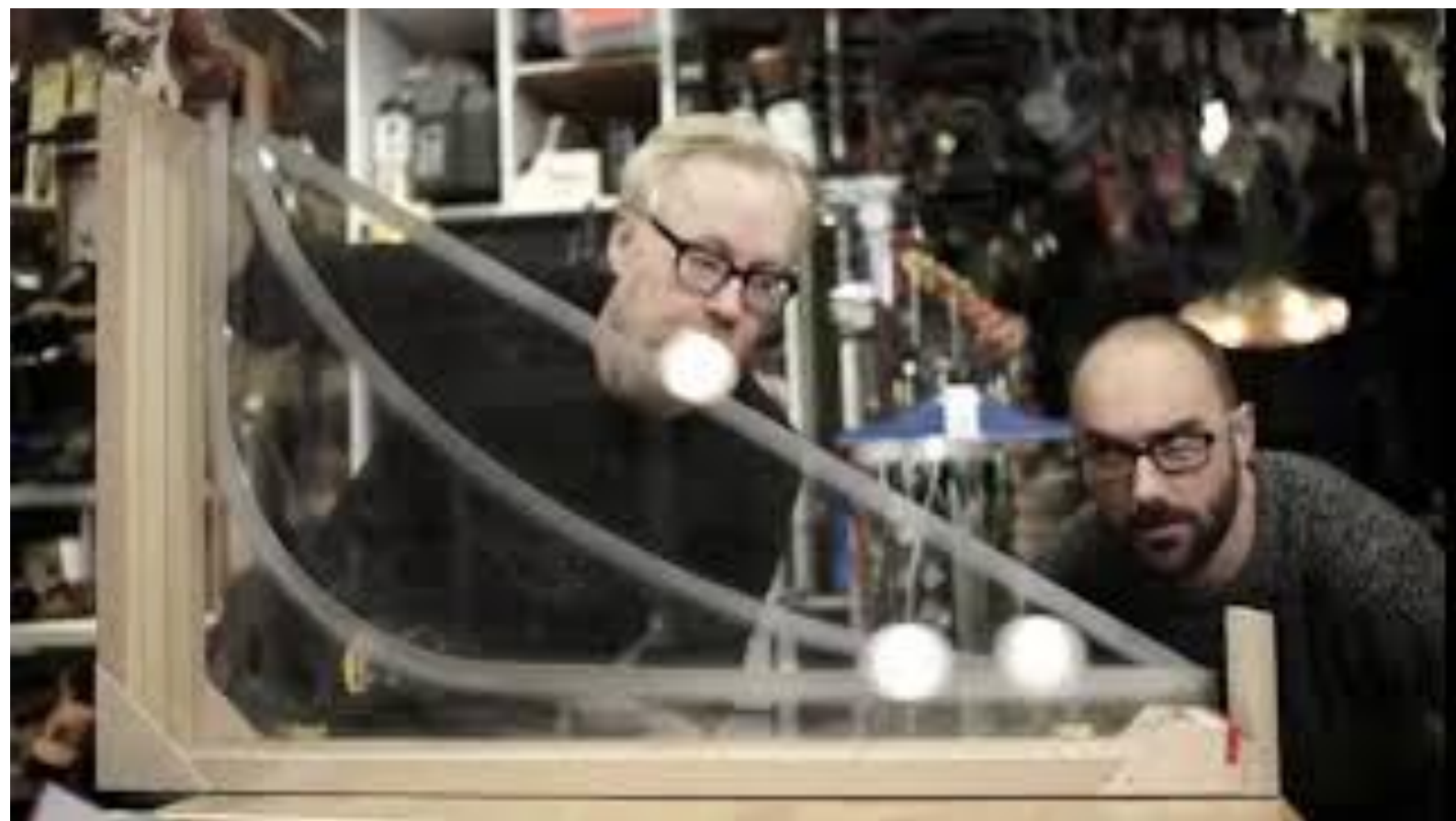
Batch Normalization

Internal Covariate Shift Problem → Moving Target Problem

Solution → “Batch Normalization”

Lecture 03: Advanced Optimizers

- Momentum
- Nesterov Accelerated Gradient (NAG)
- AdaGrad
- RMSprop
- Adam
- AdamW



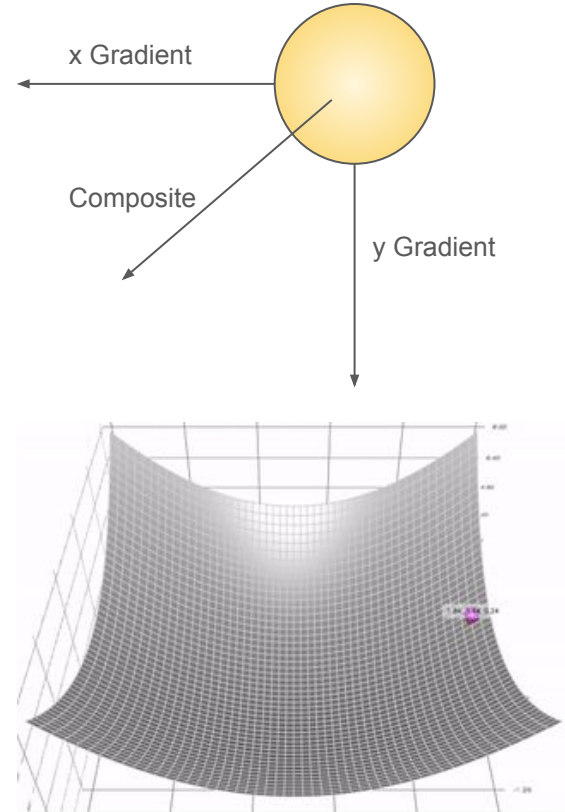
Momentum: From Physics to Machine Learning

Feature is pulling GD toward itself.

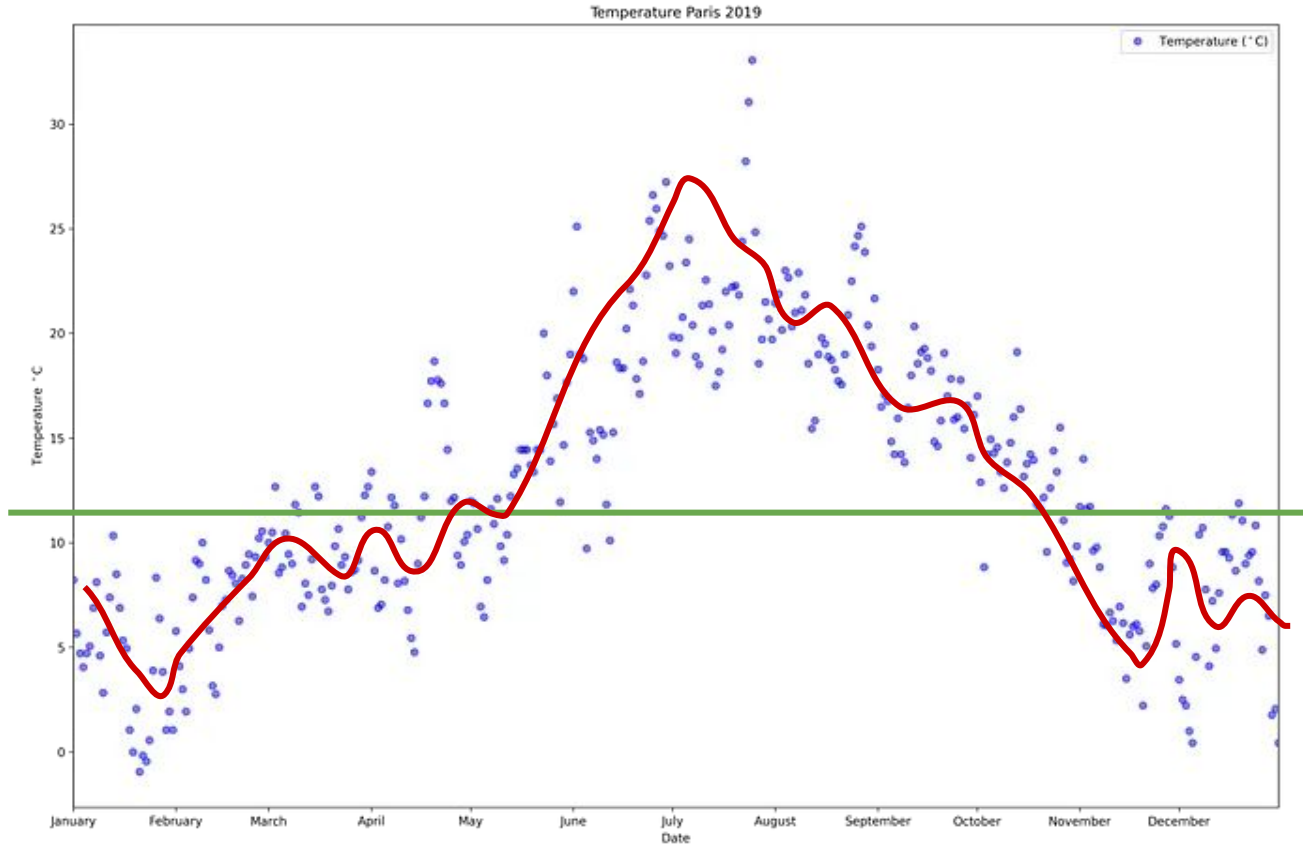
We can take composite of feature vectors

Momentum simply moves faster (because of all the momentum it accumulates)

Momentum has a shot at escaping local minima (because the momentum may propel it out of a local minimum). In a similar vein, as we shall see later, it will also power through plateau regions better.



Moving Average: Base for New Optimizers



Exponentially Weighted Moving Average (EWMA)

Exponentially Weighted Moving Average (EWMA): Technique to find trends in time series based data

Used in - Time Series Forecasting, Financial Forecasting, Signal Processing, Base of many Deep Learning Optimizers (Gradient Descent with Momentum, AdaGrad, RMSprop, Adam etc.)

Fast, Low-Memory cost

Points to remember, while calculating EWMA -

- Latest Data Weightage $>$ Older Data Weightage
- As time passes, data point weightage will be keep on reducing

Maths of EWMA

$$W_t = \begin{cases} 0 & t = 0 \\ \beta \cdot W_{t-1} + (1 - \beta) \cdot \theta_t & t > 0 \end{cases}$$

β : decides how important the current observation is, while calculating EWMA.

In general to compute the EWA for a given weight parameter, we use $0 < \beta < 1$

Let's take $\beta = 0.9$ (why 0.9 ?)

Index	Temp(Θ)	W_t	W_t
0	23	$W_0 = 0 \parallel \Theta_0$	23 (we prefer)
1	19	$W_1 = \beta W_0 + (1 - \beta) \Theta_1$	1.9
2	17	$W_2 = \beta W_1 + (1 - \beta) \Theta_2$	1.8
3	33	$W_3 = \beta W_2 + (1 - \beta) \Theta_3$	2.8
4	39	$W_4 = \beta W_3 + (1 - \beta) \Theta_4$	3.4

Example

β = Weight parameter

θ_t = Temperature day t

W_t = EWA for day t (set $W_0 = 0$)

Let $\beta = 0.9$, then

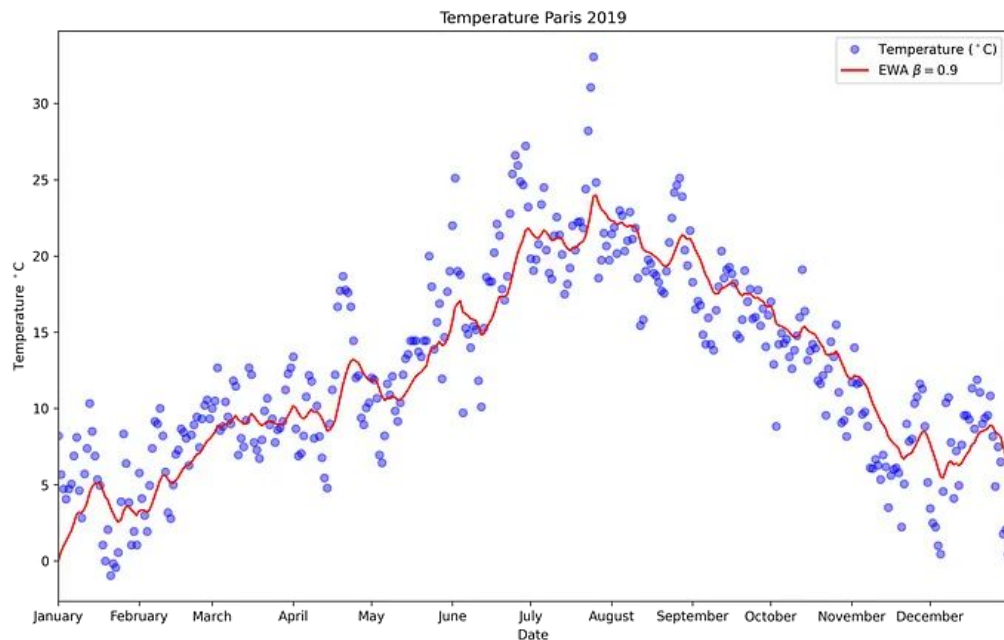
$$W_1 = 0.9 \cdot W_0 + 0.1 \cdot \theta_1$$

$$W_2 = 0.9 \cdot W_1 + 0.1 \cdot \theta_2$$

$$\vdots$$

$$W_t = 0.9 \cdot W_{t-1} + 0.1 \cdot \theta_t$$

If we plot this in red, we can see that what we get is a moving average of the daily temperature, it's like a smooth, less noisy curve.

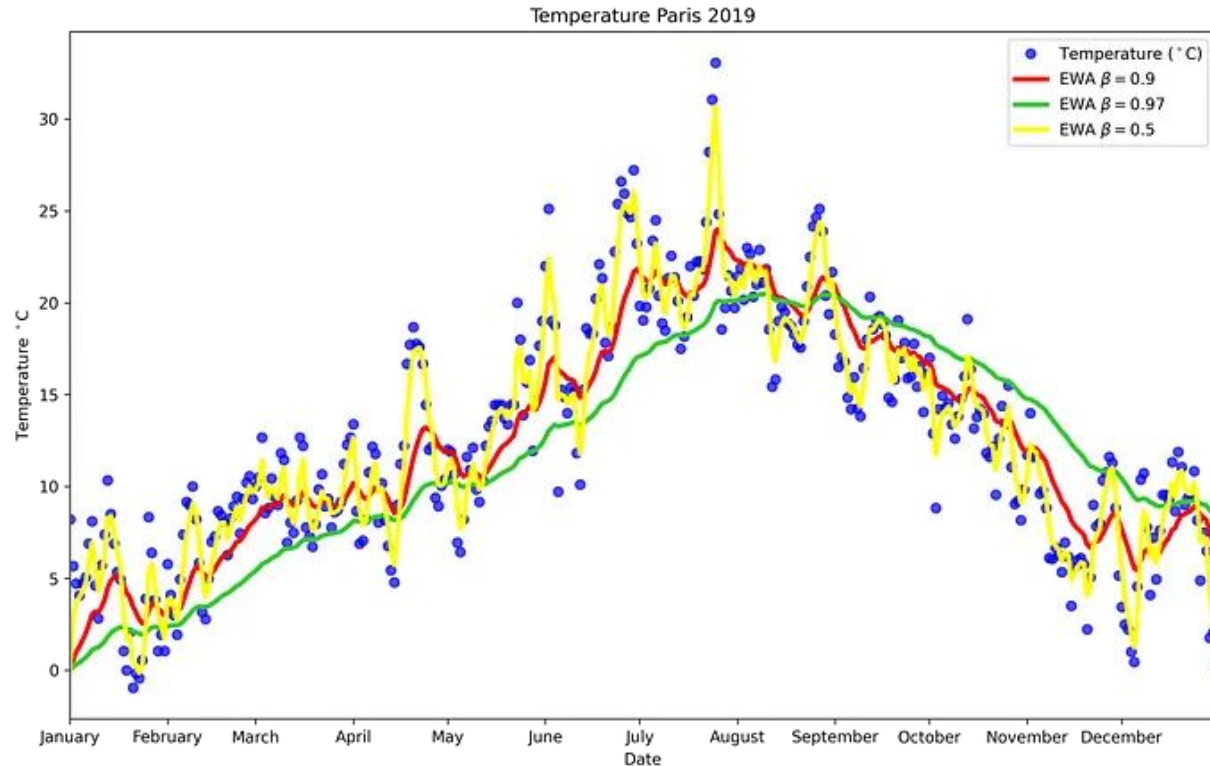


General Equation and Effect of β value

$$W_t = \underbrace{\beta \cdot W_{t-1}}_{\text{trend}} + \underbrace{(1 - \beta) \cdot \theta_t}_{\text{current value}}$$

The numbers of observations used to adapt your EWA is calculated by : $n_\beta = \frac{1}{1 - \beta}$

β	n_β	EWA
0.9	10	Adapts normal
0.98	50	Adapts slowly
0.5	2	Adapts quickly



$$V_0 = 0$$

$$V_1 = \beta V_0 + (1-\beta)\theta_1$$

$$V_2 = \beta V_1 + (1-\beta)\theta_2$$

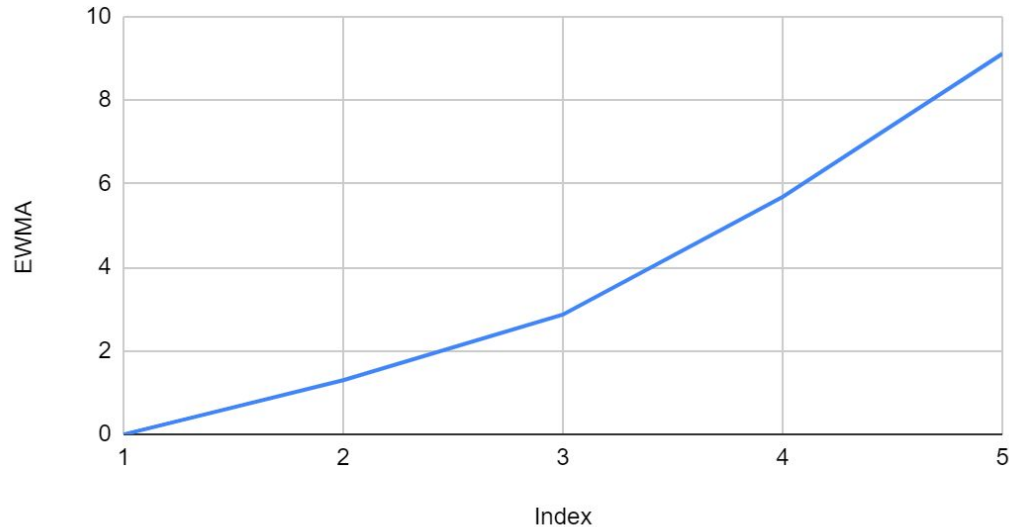
$$V_2 = \beta [\beta V_0 + (1-\beta)\theta_1] + (1-\beta)\theta_2$$

$$V_3 = \beta V_2 + (1-\beta)\theta_3$$

$$= (1-\beta)[\beta^2\theta_1 + \beta\theta_2 + \theta_3]$$

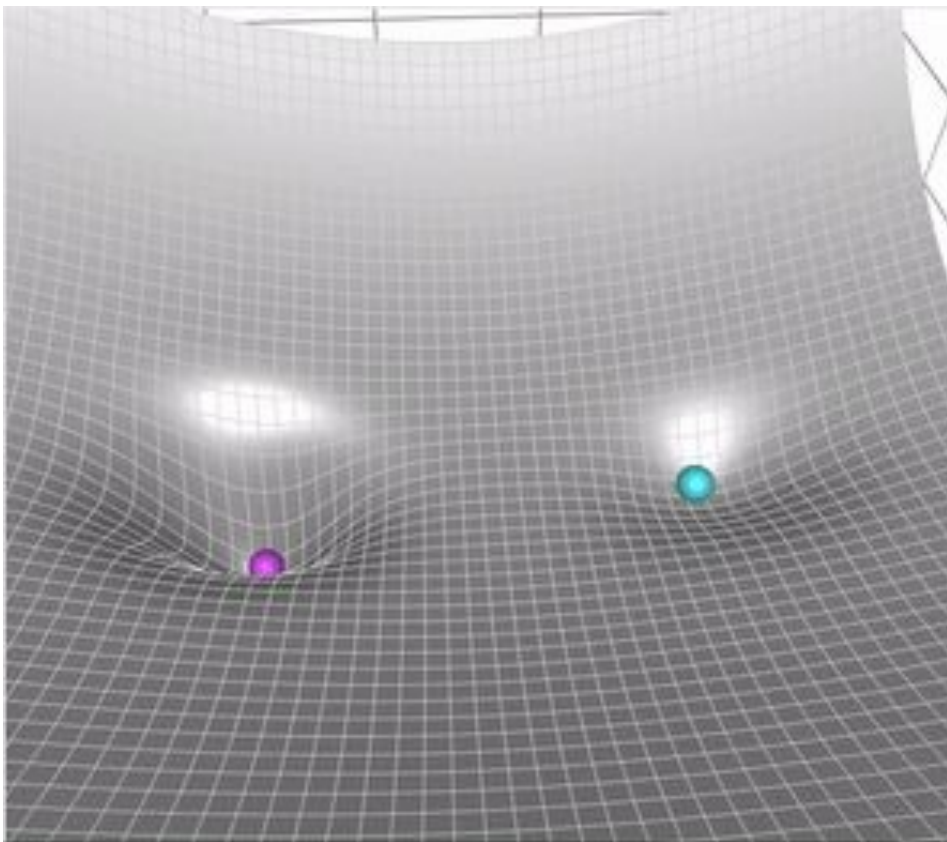
$\beta = 0.9$ so, $\beta^3 < \beta^2 < \beta$, older values weight is decreasing as compare to newer one

EWMA vs. Index



[Try Code](#)

Working of SGD with Momentum



Gradient Descent (SGD) with Momentum

The update rule for gradient descent with momentum involves a momentum coefficient (usually denoted by beta, β) between 0 and 1. It determines the weightage given to past gradients. Here's a simplified representation:

$$\text{weight_update} = \beta * \text{previous_weight_update} - \text{learning_rate} * \text{current_gradient}$$

Overall, gradient descent with momentum is a powerful technique for improving the efficiency and effectiveness of gradient descent optimization in various machine learning applications.

When $\beta = 0$ then it will work like normal SGD

Issue: Oscillate in Global Minima Solution: NAG

Nesterov Accelerated Gradient

SGD with Momentum: New Weight = History of Velocity + Gradient at that point

The idea of NAG is to measure the gradient of the cost function not at the local position but slightly ahead in the direction of the momentum.

NAG: New Weight = History of Velocity → **New Position**, then at new position Gradient will be calculated

Looking Ahead: Estimates a future position of the parameter based on the current gradient and momentum.

Benefits of NAG:

Converges Faster: It takes fewer iterations to reach the minimum of the loss function.

Handles Oscillations Better: In uneven loss landscapes with valleys and ridges, NAG avoids getting stuck in oscillations and navigates more smoothly.

NAG:

```
optimizer = keras.optimizers.SGD(lr=0.001, momentum=0.9, nesterov=True)
```

$$W_{la} = W_t - \beta V_{t-1}$$

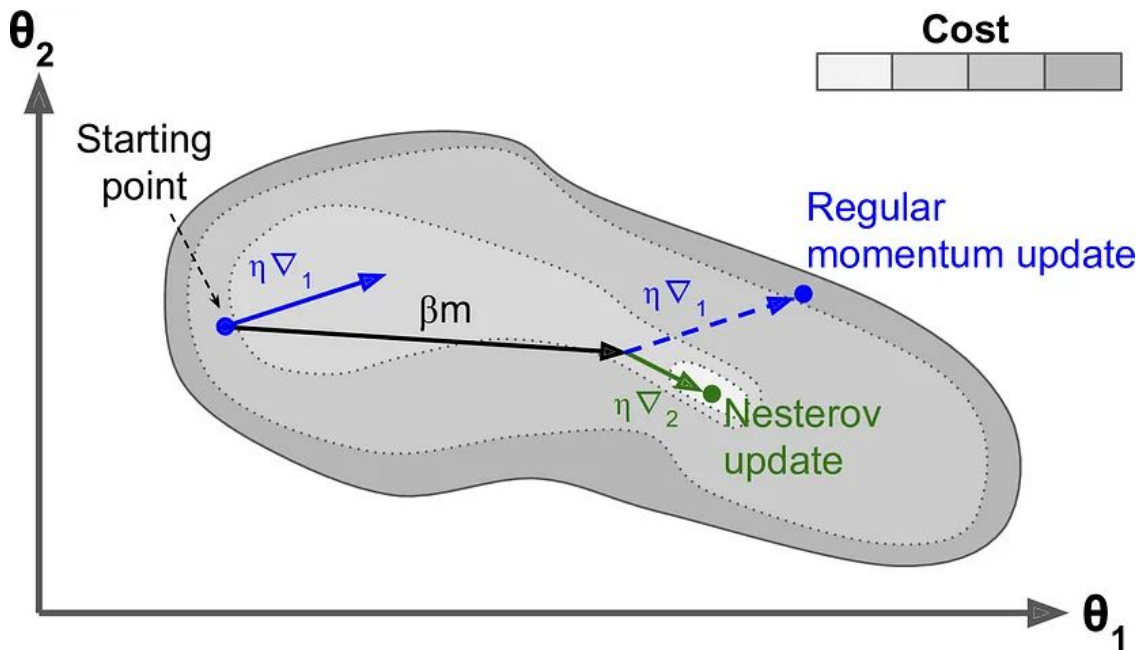
momentum

$$V_t = \beta V_{t-1} + \eta \nabla W_{la}$$

$$W_{t+} = W_t - V_t$$

Distance
between
old and
new
position

$$V_t = (W_t - W_{la}) + \eta \nabla W_{la}$$



Movement: The Math

In standard SGD, the update depends *only* on the current gradient. In Momentum, we introduce a new variable v (velocity).

$$\theta_t = \theta_t - V_t$$

$$V_t = \beta \cdot V_{t-1} + \eta \cdot \nabla \theta_t$$

Nesterov Accelerated Gradient (NAG)

There is a famous improvement to Momentum called **Nesterov Momentum**.

- **The Flaw in Standard Momentum:** The "heavy ball" might roll so fast that it misses the minimum and rolls up the other side before slowing down.
- **The Nesterov Fix:** It effectively "looks ahead." It calculates the gradient not at the current position, but at the position where the momentum is *about* to take us.
 - *Analogy:* A smart driver doesn't brake when they hit the stop sign; they brake *before* they get there because they see it coming.

NAG: Math

$$\theta_{la} = \theta_t - \beta \cdot V_{t-1}$$

$$V_t = \beta \cdot V_{t-1} + \eta \cdot \nabla \theta_{la}$$

Adaptive Gradient: AdaGrad

Issues with NAG:

Case 1: Features scales differently Solution → Normalization

Case 2: Features sparse in nature → Elongated Bowl Problem

Solution: AdaGrad

AdaGrad: keeps track of the sum of gradient squared and uses that to adapt the gradient in different directions.

AdaGrad will take a straight path, whereas gradient descent (or relatedly, Momentum) takes the approach of “let me slide down the steep slope first and maybe worry about the slower direction later”. Sometimes, vanilla gradient descent might just stop at the saddle point where gradients in both directions are 0 and be perfectly content there.

AdaGrad

Out of $W = W - \eta * \partial L / \partial w$ and $b = b - \eta * \partial L / \partial b$

One is updating fast other very slow, so all we can adjust is learning rate

Update  learning rate 

Different learning rate for different feature is idea of AdaGrad

$\eta / \sqrt{V_t + \epsilon}$, ϵ is very small +ve no.

$$V_t = V_{t-1} + (\partial L / \partial w)^2$$

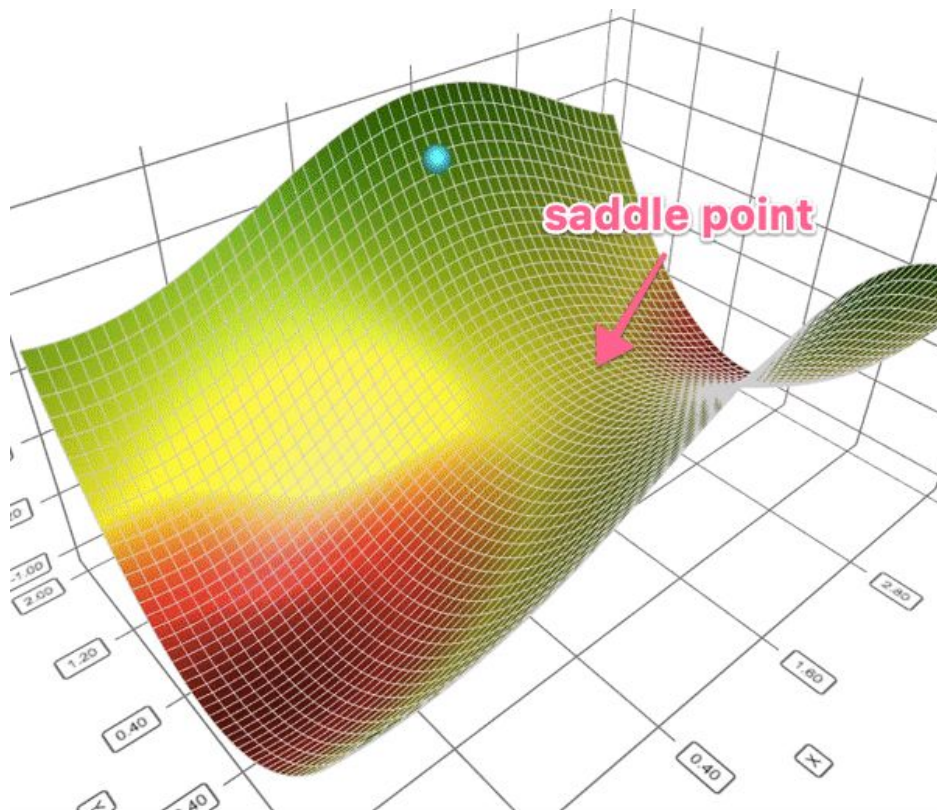
AdaGrad: Math

$$\theta_t = \theta_t - \eta_t' \cdot V_t$$

$$\eta_t' = \eta / \sqrt{V_t} + \varepsilon$$

$$V_t = (\nabla \theta_{t-1})^2$$

Working of AdaGrad



RMSProp

Issues with AdaGrad:

As V_t becoming larger, $\eta \rightarrow 0$, Update becomes smaller, may get trap in local minima, may not reach to solution

Good for convex problems but not for non-convex problem

Solution: RMSProp and Adam

RMSProp: RootMeanSquare Prop. improvement over AdaGrad by combining EWMA and Exponential Decaying Average

$$V_t = \beta V_{t-1} + (1 - \beta) (\nabla W_t)^2 \quad \text{Initial } V_0 = 0, \beta = 0.95$$

RMSProp: Math

Cold Start: Because V_t is artificially close to zero (due to the cold start bias), the denominator $\sqrt{V_t}$ becomes tiny. Dividing by a tiny number results in a massive value. Consequently, RMSProp's initial updates can be aggressively large, potentially shooting the weights far away from good regions or even causing immediate divergence (NaNs) if the epsilon ϵ isn't set high enough to cushion the blow.

$$\theta_t = \theta_{t-1} - \eta / \sqrt{V_t} + \epsilon * G_t$$

$$G_t = \nabla J(\theta_t)$$

$$V_t = \beta V_{t-1} + (1-\beta) G_t^2$$

AdaDelta

Issue with RMSProp: diminishing learning rates, Solution → AdaDelta

AdaDelta uses an exponentially decaying average of past squared gradients and past squared updates to adaptively adjust the learning rate during training.

Uses Δx (Accumulated Update) to calculate learning rate, initially $\infty = 0$ and $\Delta x = 0$

To avoid infinite increase in ∞ , introduces ρ (Rho) as a decay constant, $\rho = 0.9$

$$\infty_t = \rho \cdot \infty_{t-1} + (1 - \rho) \cdot g_t^2$$

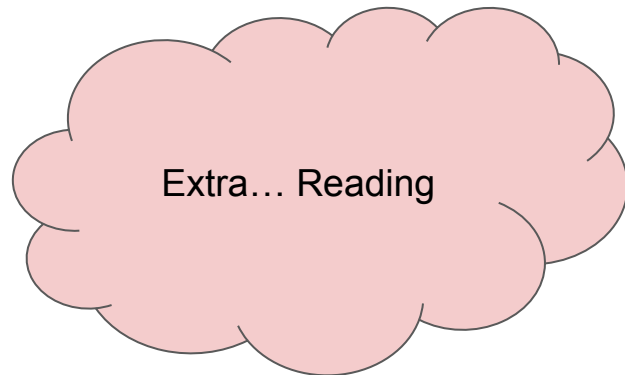
$$\Delta x_t = \rho \cdot \Delta x_{t-1} + (1 - \rho) \cdot \Delta \Theta_t^2$$

$$\Delta \Theta_{t+1} = \Theta_t + \Delta \Theta_t$$

$$\Delta \Theta_t = \sqrt{(\Delta x_t + \epsilon) / (\infty_t + \epsilon)} \cdot g_t$$

Learning Rate

$$(\partial L / \partial w)^2$$



Adam: Math

Cold Start solved by Bias Correction

But

Wrongly implemented regularization

$$\theta_t = \theta_{t-1} - \eta / \sqrt{Vt} + \varepsilon * M_{t-1}$$

$$Mt = \beta_1 M_{t-1} + (1 - \beta_1) * Gt$$

$$Vt = \beta_2 M_{t-1} + (1 - \beta_2) * Gt^2$$

Bias correction with $t = \text{epoch \#}$

$$\hat{M}t = Mt / (1 - \beta_1^t)$$

$$\hat{V}t = Vt / (1 - \beta_2^t)$$

$$\beta_1 = 0.9$$

$$\beta_2 = 0.99$$

AdamW: Adam with Weight Decay

1. $g_t = \nabla J(\theta_t) + \lambda \theta_t$
L2 is mixed into gradient
2. Calculate $\widehat{m}_t, \widehat{v}_t$ based on g_t
3. $\theta_{t+1} = \theta_t - \eta \frac{\widehat{m}_t}{\sqrt{\widehat{v}_t + \epsilon}}$

1. $g_t = \nabla J(\theta_t)$
Pure gradient only
2. Calculate $\widehat{m}_t, \widehat{v}_t$ based on g_t
3. $\theta_{t+1} = \theta_t - \eta \left(\frac{\widehat{m}_t}{\sqrt{\widehat{v}_t + \epsilon}} + \lambda \theta_t \right)$



Optimizers Summary

Learning Rate Scheduling: The Common Schedules

A. Step Decay

This is the simplest method. You drop the learning rate by a specific factor (e.g., 0.1) after a set number of epochs.

- **Mechanism:** e.g. "Divide LR by 10 every 30 epochs."
- **Visual:** It looks like a staircase.
- **Use Case:** Standard computer vision tasks (e.g., ResNet on ImageNet) often use this. It is rigid but effective if you know exactly how long your model needs to train.

Learning Rate Scheduling: The Common Schedules

B. ReduceLROnPlateau (Adaptive)

This is a "smart" scheduler. It watches a metric (like Validation Loss). If the loss stops going down for a while (patience), it reduces the LR.

- **Mechanism:** "If val_loss doesn't improve for 5 epochs, cut LR by half."
- **Visual:** It looks like a staircase, but the steps happen at irregular intervals based on model performance.
- **Use Case:** When you don't know the optimal training time. It is very popular because it adapts to the specific run.

Learning Rate Scheduling: The Common Schedules

C. Cosine Annealing

This follows a cosine curve. It starts high, drops slowly at first, then drops rapidly in the middle, and finally slows down again as it approaches zero.

- **Mechanism:**

$$LR_t = \frac{1}{2} \left(1 + \cos \left(\frac{t\pi}{T} \right) \right)$$

- **Visual:** A smooth wave shape.
- **Use Case:** Very popular in modern research (especially with AdamW). The smooth transition often yields better results than the sharp drops of Step Decay.

Learning Rate Scheduling: Modern Techniques

A. Warm-up

In modern architectures like Transformers (BERT, GPT), the gradients can be very unstable at the very beginning of training (the "cold start" problem we discussed with Adam). If you start with a high LR immediately, the model might diverge.

- **Solution:** Start with a tiny LR (e.g., 0) and linearly increase it to your target LR over the first few thousand steps. *Then* start your decay schedule.

Learning Rate Scheduling: Modern Techniques

B. Cyclic Learning Rates (CLR) & OneCycle

Instead of just going down, these schedules make the LR go **up and down**.

- **The Idea:** Periodically increasing the LR helps the model jump out of "sharp" local minima (bad solutions) and settle into "flat" minima (stable solutions that generalize better).
- **OneCycle Policy:** A specific variation where the LR goes up for the first 30% of training and down for the remaining 70%. It is known as a "super-convergence" technique because it allows models to train much faster.
- [Code](#)

Solutions for Overfitting

1. Add more data
2. Reduce Complexity
3. Regularization (L1 , L2)
4. Early Stopping
5. Dropout
6. Greedy Layer-wise Pre-Training

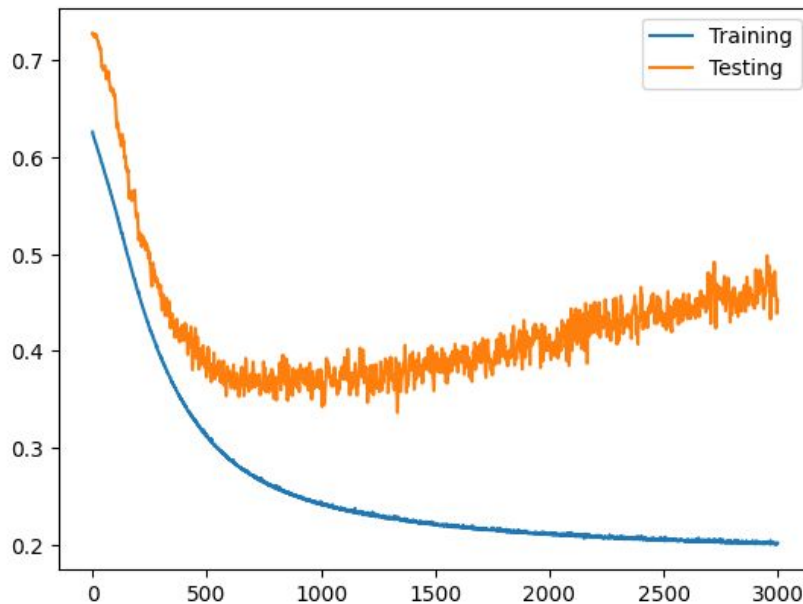
Early Stopping

More is better...?

How to tune...?

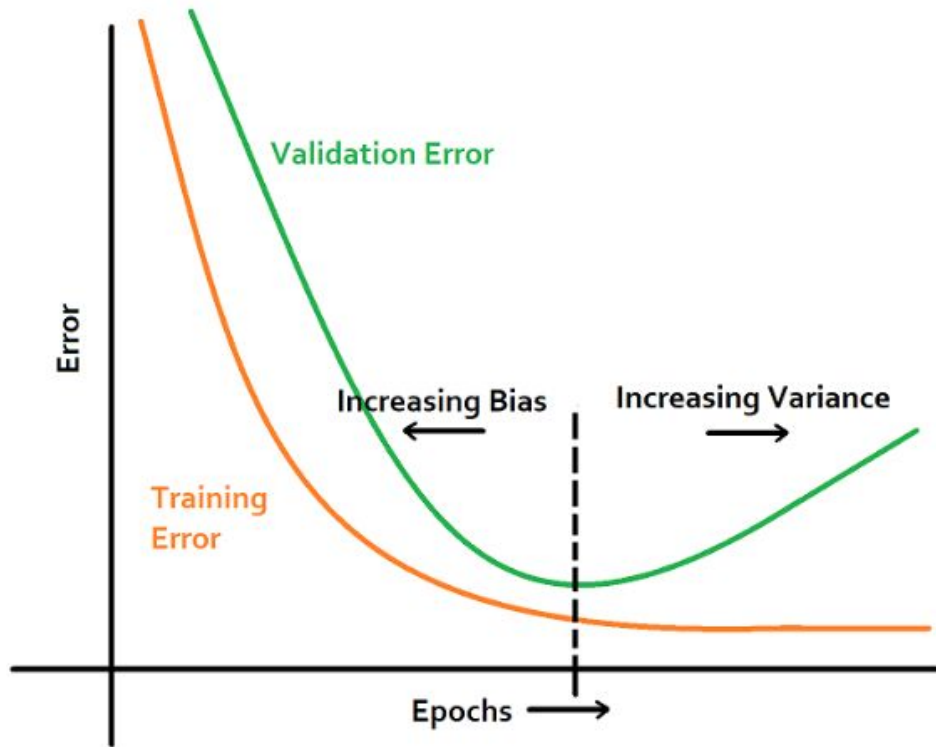
Early Stopping

- Useful to avoid overfitting
- Setting proper no. of epochs



Early Stopping

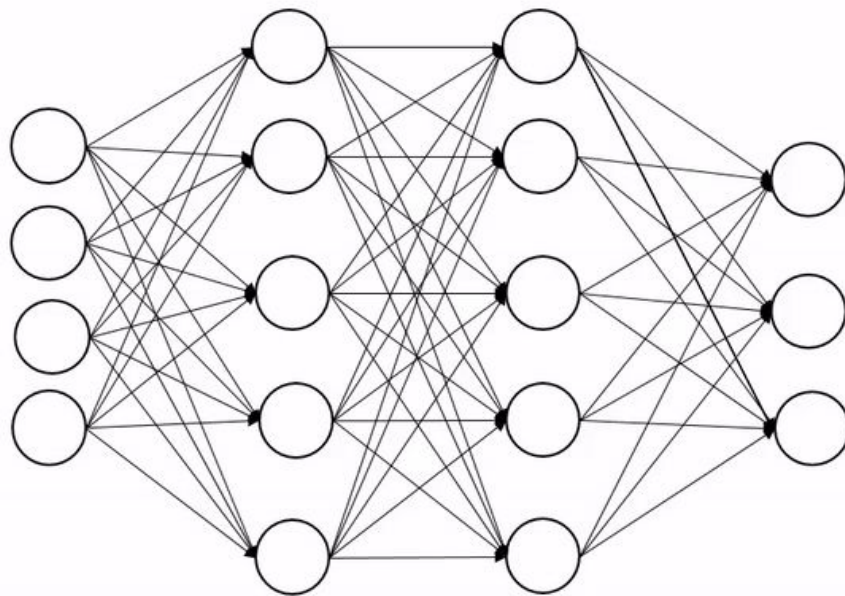
```
tf.keras.callbacks.EarlyStopping(  
    monitor="val_loss",  
    min_delta=0,  
    patience=0,  
    verbose=0,  
    mode="auto",  
    baseline=None,  
  
    restore_best_weights=False,  
    start_from_epoch=0,  
)
```



Dropouts

“Dropout” refers to the practice of disregarding certain nodes in a layer at random during training. A dropout is a regularization approach that prevents overfitting by ensuring that no units are codependent with one another.

Dropout makes the training process noisy, requiring nodes within a layer to take on more or less responsible for the inputs on a probabilistic basis.



Dropouts

@ every epoch some of the neurons will be dropped

@ every epoch dropout rate (P) can be set different

Dropout happens only during training phase

Dropout DOES NOT happens during testing phase

We need to adjust the weights according to dropout rate - dropout probability

`model.add(Dropout(P))`

How to use...?

- Overfitting $\rightarrow P \uparrow$
- Underfitting $\rightarrow P \downarrow$
- Apply dropout at last layer, then try for earlier layers
- ANN $\rightarrow 10-50\%$
- RNN $\rightarrow 20-30\%$
- CNN $\rightarrow 40-50\%$

Drawbacks of Dropout

Convergence Delay

Loss function value changes

Gradient calculation becomes difficult while debugging